

# claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_5d655769-ba4\agent-a9194f5c6ed999a7f.jsonl`  
- SHA-256: `02ff868dba177f194081d49fbb07a0304f31a57d33e5f9933ec582845e93d79d`  
- Source modified: `2026-06-21T17:20:16+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_5d655769-ba4`  
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

## Transcript

### 1. user (2026-06-21T17:17:44.236Z)

Review A9 for UX + faithfulness + a11y. A9 (tracker FRIENDLY\_AND\_MULTILINGUAL \$7): localized tap-only filter controls so a non-English-typing user can filter without typing the English query syntax.

Files under C:/Users/avidu/Projects/khelsutra/web/src: NEW components/TapFilters.tsx (exports TAP\_FILTERS data + the component); App.tsx renders <TapFilters onApply={onApplyQuery}/> below <QueryBar/>; catalogs en/kn/hi/es/da/id gained a tap.\* block (title/longest/shortest/over10/over20/top5/top10). Each control builds a ParsedQuery DIRECTLY (no parsing) and calls the shared onApply(q,"example"). A test pins tap==type (parseQuery(f.english) deep-equals f.parts). 173 web tests green; tsc+build clean; browser-verified (tap "Over 10 seconds" ? 3 rallies; labels localize).

Read C:/Users/avidu/Projects/khelsutra/web/src/components/TapFilters.tsx +

C:/Users/avidu/Projects/khelsutra/web/src/components/QueryBar.tsx + the en catalog tap.\* block.

1. REDUNDANCY: QueryBar still renders English EXAMPLE chips ("over 10s" etc.) under "Sample queries:", and now TapFilters renders localized chips under "Or tap to filter:". Two chip rows ? is this acceptable, or confusing clutter for a non-tech user? Concrete recommendation (keep both / merge / differentiate copy).

2. a11y: buttons are ?44px (min-h-11), real <button>, localized text labels (icon-free, fine).

Any a11y gap? Does the no-bare-strings hygiene test cover TapFilters (it was added to the FILES list)?

3. FAITHFULNESS: does A9 meet "tap-only filter controls alongside the English query bar so kn/hi users can filter without typing" (\$6.5)? Is anything essential missing (e.g. a way to clear/reset ? though bulk Select-all exists)?

4. Copy: are the tap.\* labels (en) clear + honest? Localized sets exist for all 6 ? any obvious issue?

Be precise. Empty findings if solid.

### 2. assistant (2026-06-21T17:17:47.903Z)

I'll review A9. Let me start by reading the relevant files.

### 3. user (2026-06-21T17:17:48.361Z)

```
1      // Localized tap-only filter controls (tracker A9). The query bar PARSES ENGLISH text (a  
2      // limit) ? so a Kannada/Hindi/? user who can't type English needs to filter by TAPPING.  
3      // builds the SAME ParsedQuery the parser would for its English equivalent ? directly,  
4      // round-trip ? and applies it through the shared onApply path. A test pins tap == type  
5      // can never silently drift from the parser. Length-led + fully-available only (never  
6      import { useTranslation } from "react-i18next";  
7      import type { ParsedQuery } from "../lib/query";
```

```

8
9     interface TapFilter {
10         /** i18n label key (localized) ? the user sees this, not the English query. */
11         key: string;
12         /** The equivalent typed query; the tap MUST select identically to typing this (pinned
by a test). */
13         english: string;
14         parts: Pick<ParsedQuery, "filters" | "sort" | "limit">;
15     }
16
17     export const TAP_FILTERS: TapFilter[] = [
18         { key: "tap.longest", english: "longest first", parts: { filters: [], sort: { key:
"duration", dir: "desc" } } },
19         { key: "tap.shortest", english: "shortest first", parts: { filters: [], sort: { key:
"duration", dir: "asc" } } },
20         { key: "tap.over10", english: "over 10s", parts: { filters: [{ field: "duration", op:
">", value: 10 } ] } },
21         { key: "tap.over20", english: "over 20s", parts: { filters: [{ field: "duration", op:
">", value: 20 } ] } },
22         { key: "tap.top5", english: "top 5", parts: { filters: [], sort: { key: "duration",
dir: "desc" }, limit: 5 } },
23         { key: "tap.top10", english: "top 10", parts: { filters: [], sort: { key: "duration",
dir: "desc" }, limit: 10 } },
24     ];
25
26     interface Props {
27         /** Same handler the QueryBar uses ? App runs runQuery + writes the shared selection.
*/
28         onApply: (q: ParsedQuery, source: "typed" | "example") => void;
29     }
30
31     export function TapFilters({ onApply }: Props) {
32         const { t } = useTranslation();
33         // ?44px tap targets (A4) so the no-type path is comfortable on a phone.
34         const BTN = "inline-flex min-h-11 items-center rounded-full border border-black/15
px-4 text-sm hover:border-green";
35
36         return (
37             <div className="mt-3" data-testid="tap-filters">
38                 <span className="text-xs text-ink/40">{t("tap.title")}</span>
39                 <div className="mt-1.5 flex flex-wrap gap-2">
40                     {TAP_FILTERS.map((f) => (
41                         <button
42                             key={f.key}
43                             type="button"
44                             className={BTN}
45                             onClick={() =>
46                                 onApply(
47                                     { raw: `tap:${f.key}`, filters: f.parts.filters, sort: f.parts.sort,
limit: f.parts.limit, unavailable: [], recognized: true },
48                                     "example",
49                                 )
50                             }
51                         >
52                             {t(f.key)}
53                         </button>
54                     ))}
55                 </div>
56             </div>
57         );
58     }
59

```

```

1      // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2      // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3      // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4      // text was understood, and concepts with no data behind them (shot-type / player /
score) are
5      // greyed out as "not available yet" ? never silently faked.
6      import { useMemo, useState } from "react";
7      import type { FormEvent } from "react";
8      import { useTranslation, Trans } from "react-i18next";
9      import { parseQuery, summarize } from "../lib/query";
10     import type { ParsedQuery } from "../lib/query";
11
12     interface Props {
13       /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
selection).
14       * `source` distinguishes a typed-and-submitted query from an example-chip tap (for
the trace). */
15       onApply: (q: ParsedQuery, source: "typed" | "example") => void;
16     }
17
18     // Each example is a real, parseable query ? tapping one fills the box AND applies it.
Led by rally
19     // LENGTH (the trusted axis); shot-count filtering exists but is experimental, so it's
not featured
20     // here (a test pins that every example parses to a recognized, fully-available query).
21     export const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "first 10"];
22
23     // Parsed-pill chips: legible on a phone (text-sm, not text-xs) ? A4.
24     const CHIP = "rounded-full px-3 py-1 text-sm font-medium";
25
26     export function QueryBar({ onApply }: Props) {
27       const { t } = useTranslation();
28       const [value, setValue] = useState("");
29       const parsed = useMemo(() => parseQuery(value), [value]);
30       const clauses = summarize(parsed);
31       const typed = value.trim().length > 0;
32
33       const applyText = (text: string) => {
34         setValue(text);
35         onApply(parseQuery(text), "example");
36       };
37       const submit = (e: FormEvent) => {
38         e.preventDefault();
39         if (parsed.recognized) onApply(parsed, "typed");
40       };
41
42       return (
43         <div>
44           <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
45             <input
46               type="text"
47               value={value}
48               onChange={(e) => setValue(e.target.value)}
49               placeholder={t("query.placeholder")}
50               aria-label={t("query.ariaLabel")}
51               className="min-h-11 min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-
white px-4 py-2 text-sm text-ink outline-none focus:border-green"
52             />
53             <button
54               type="submit"
55               disabled={!parsed.recognized}
56               className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-

```

```

white disabled:opacity-40"
57         >
58         {t("query.apply")}
59     </button>
60 </form>
61
62     {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
*/}
63     <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
items-center gap-1.5">
64         {clauses.map((c) => (
65             <span key={c} className={CHIP + " bg-green/15 text-green"}>
66                 {c}
67             </span>
68         ))}
69         {parsed.unavailable.map((u) => (
70             <span
71                 key={u.token}
72                 title={t("query.notAvailableTitle", { token: u.token, kind: u.kind })}
73                 className={CHIP + " bg-black/5 text-ink/45"}
74             >
75                 <span className="line-through">{u.token}</span> · {t("query.notAvailable")}
76             </span>
77         ))}
78         {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
79             <span className="text-xs text-ink/50">{t("query.unreadable")}</span>
80         )}
81         {!typed && <span className="text-xs text-ink/50">{t("query.hint")}</span>}
82     </div>
83
84     {/* Surface the "not available yet" reason as VISIBLE text (the per-chip title is
hover-only,
85         so it never appears on a phone) ? A4. */}
86     {parsed.unavailable.length > 0 && (
87         <p className="mt-1.5 text-xs text-ink/50">{t("query.notAvailableHelp")}</p>
88     )}
89
90     <div className="mt-3 flex flex-wrap items-center gap-2 text-sm">
91         <span className="text-ink/40">{t("query.try")}</span>
92         {EXAMPLES.map((ex) => (
93             <button
94                 key={ex}
95                 type="button"
96                 onClick={() => applyText(ex)}
97                 className="inline-flex min-h-11 items-center rounded-full border border-
black/15 px-4 text-sm hover:border-green"
98             >
99                 {ex}
100             </button>
101         ))}
102     </div>
103
104     <p className="mt-2 text-xs text-ink/40">
105         <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60"
/>, x: <span className="text-amber-700" /> }} />
106     </p>
107 </div>
108 );
109 }
110

```

## 5. assistant (2026-06-21T17:17:54.528Z)

Now let me read the en catalog tap.\* block and find the i18n catalogs.

## 6. user (2026-06-21T17:17:54.717Z)

```
1      import { useEffect, useMemo, useState } from "react";
2      import { useTranslation, Trans } from "react-i18next";
3      import { useRallySelection } from "../hooks/useRallySelection";
4      import { RallyGrid } from "../components/RallyGrid";
5      import { QueryBar } from "../components/QueryBar";
6      import { TapFilters } from "../components/TapFilters";
7      import { SelectionFooter } from "../components/SelectionFooter";
8      import { LanguageSwitcher } from "../components/LanguageSwitcher";
9      import { CompileResult, type BuildState } from "../components/CompileResult";
10     import { IngestPanel } from "../components/IngestPanel";
11     import { runQuery, type ParsedQuery } from "../lib/query";
12     import { floorRisk } from "../lib/floor";
13     import { reelFilename } from "../lib/format";
14     import { classifyError } from "../lib/errors";
15     import { errorText } from "../lib/errorText";
16     import { log, newCorrelationId } from "../lib/log";
17     import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
18     import type { EngineConfig, RallySegment } from "../api/types";
19
20     /** One compact, structured description of how a query was understood (for the trace
log). */
21     function parseTrace(q: ParsedQuery) {
22         return {
23             raw: q.raw,
24             recognized: q.recognized,
25             filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
26             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
27             limit: q.limit ?? null,
28             unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
29         };
30     }
31
32     // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
33     // contract; only honest fields are surfaced. A real session loads from POST /api/query.
34     const DEMO_RALLIES: RallySegment[] = [
35         { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
36         { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
37         { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
38         { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
39         { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
40         { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
41     ];
42
43     type LoadState =
44         | { status: "demo" }
45         | { status: "loading" }
46         | { status: "loaded"; count: number }
47         | { status: "error"; error: string };
48
49     export default function App() {
50         const { t } = useTranslation();
51         // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
52         // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
```

```

53 // without a reload (it also rewrites ?video= so a refresh is stable).
54 const [videoId, setVideoId] = useState<string | null>()
55   () => new URLSearchParams(window.location.search).get("video"),
56   );
57
58 const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
59 const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
60 const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
61 const [reelName, setReelName] = useState("highlights");
62 const [build, setBuild] = useState<BuildState>({ status: "idle" });
63
64 // Display order is separate from selection: a query can re-sort the cards while the
shared
65 // selection set (sel) stays the single source of truth for what lands in the reel.
66 const sel = useRallySelection(rallies, "all");
67
68 // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
69 useEffect(() => {
70   if (!videoId) return;
71   const cid = newCorrelationId();
72   log("info", "rallies.load_start", { cid, video_id: videoId });
73   // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
74   // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
75   setLoadState({ status: "loading" });
76   let alive = true;
77   listRallies(videoId, cid)
78     .then((rs) => {
79       if (!alive) return;
80       setRallies(rs);
81       setOrder(rs);
82       sel.apply(rs.map((r) => r.id));
83       setLoadState({ status: "loaded", count: rs.length });
84       log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
85     })
86     .catch((err) => {
87       if (!alive) return;
88       // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
89       // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
90       const e = classifyError(err);
91       setLoadState({ status: "error", error: errorText(t, e) });
92       log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
93     });
94   return () => {
95     alive = false;
96   };
97   // videoId is stable for the page; sel.apply is a stable callback.
98   // eslint-disable-next-line react-hooks/exhaustive-deps
99 }, [videoId]);
100
101 // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
102 // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
103 const [cfg, setCfg] = useState<EngineConfig | null>(null);
104 useEffect(() => {
105   let alive = true;
106   getConfig()
107     .then((c) => alive && setCfg(c))
108     .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
109   return () => {

```

```

110         alive = false;
111     };
112 }, []);
113 const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
114
115 // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
116 const ordinalOf = useMemo(() => {
117     const m = new Map<number, number>();
118     [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
119 m.set(r.id, i + 1));
120     return (id: number) => m.get(id) ?? 0;
121 }, [rallies]);
122
123 // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
124 whole call.
125 const onBuild = async () => {
126     if (build.status === "building") return;
127     const cid = newCorrelationId();
128     const filename = reelFilename(reelName);
129     const ids = [...sel.selected];
130     if (!videoId) {
131         log("info", "build.no_video", { cid, selected: ids.length });
132         setBuild({
133             status: "error",
134             error: t("build.noVideo"),
135         });
136         return;
137     }
138     setBuild({ status: "building" });
139     log("info", "build.start", {
140         cid,
141         video_id: videoId,
142         filename,
143         selected: ids.length,
144         total_sec: Math.round(sel.totalSec),
145         below_floor: warning?.atRisk.length ?? 0,
146     });
147     try {
148         const res = await compileReel(videoId, ids, filename, cid);
149         const url = highlightUrl(videoId, filename);
150         setBuild({ status: "done", url, filename });
151         log("info", "build.done", { cid, video_id: videoId, filename, filepath:
152 res.filepath, url });
153     } catch (err) {
154         // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
155         "is the engine
156         // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
157         (A5).
158         const status = err instanceof ApiError ? err.status : undefined;
159         const e = classifyError(err);
160         setBuild({ status: "error", error: errorText(t, e) });
161         log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
162 e.code, detail: e.detail, error: String(err) });
163     }
164 };
165
166 // Every query application is ONE correlatable journey: a `query.parse` line records
167 exactly how
168 // the text was understood, then a single outcome line (apply / skip / empty) shares
169 the same cid.
170 // Unsupported concepts and zero-result queries are logged loudly ? never silently
171 dropped.
172 const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
173     const cid = newCorrelationId();
174     log("info", "query.parse", { cid, source, ...parseTrace(q) });

```

```

166
167 // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
168 // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
169 if (q.unavailable.length > 0) {
170   log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
171 }
172
173 if (!q.recognized) {
174   log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
175   return; // empty / unparseable ? leave the user's curation untouched
176 }
177
178 try {
179   const { ordered, selectedIds } = runQuery(rallies, q);
180   setOrder(ordered);
181   sel.apply(selectedIds);
182   log("info", "query.apply", {
183     cid,
184     source,
185     filters: q.filters.length,
186     sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
187     limit: q.limit ?? null,
188     total: rallies.length,
189     selected: selectedIds.length,
190     selected_ids: selectedIds,
191     dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
192   });
193   if (selectedIds.length === 0) {
194     // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
195     log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
196   }
197 } catch (err) {
198   // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
199   log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
200 }
201 };
202
203 // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
204 const onToggle = (id: number) => {
205   log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
206   sel.toggle(id);
207 };
208 const onBulk = (op: "select_all" | "clear" | "invert") => {
209   const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
210   log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
211   if (op === "select_all") sel.selectAll();
212   else if (op === "clear") sel.clearAll();
213   else sel.invert();
214 };
215
216 return (
217   <main className="min-h-screen bg-paper text-ink pb-24">
218     <header className="bg-ink text-white">
219       <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
220         <span className="text-xl font-extrabold tracking-tight">

```

```

221         Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
222         </span>
223         <LanguageSwitcher />
224     </div>
225 </header>
226
227     <section className="mx-auto max-w-5xl px-6 py-10">
228         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
229             {t("app.badge")}
230         </span>
231         <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
232         <p className="mt-2 max-w-2xl text-ink/70">
233             <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
234         </p>
235
236         {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
237         path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
238         {!videoId && (
239             <IngestPanel
240                 onLoad={id => {
241                     setVideoId(id);
242                     window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
243                 }}
244             />
245         )}
246
247         <ModeBanner load={loadState} videoId={videoId} />
248
249         <div className="mt-6">
250             <QueryBar onApply={onApplyQuery} />
251             {/* A9: localized tap controls for users who can't type the English query
syntax. */}
252             <TapFilters onApply={onApplyQuery} />
253         </div>
254
255         <div className="mt-6 flex flex-wrap gap-2 text-sm">
256             <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
257             <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
258             <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
259         </div>
260
261         <div className="mt-5">
262             <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
263         </div>
264
265         <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
266     </section>
267
268     <SelectionFooter
269         count={sel.count}
270         totalSec={sel.totalSec}
271         reelName={reelName}
272         onReelNameChange={setReelName}

```

```

273         onBuild={onBuild}
274         building={build.status === "building"}
275         warning={warning}
276     />
277 </main>
278 );
279 }
280
281 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
282     if (load.status === "demo") {
283         return (
284             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
285                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
286             </p>
287         );
288     }
289     if (load.status === "loading") {
290         return (
291             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
292                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
293             </p>
294         );
295     }
296     if (load.status === "error") {
297         return (
298             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
299             text-red-800" data-testid="mode-banner">
300                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
301             components={{ code: <code /> }} />
302             </p>
303         );
304     }
305     return (
306         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
307             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
308             <strong className="text-ink/70" /> }} />
309         </p>
310     );
311 }

```

## 7. user (2026-06-21T17:17:54.837Z)

```

web\src\lib\log.ts
web\src\lib\selection.test.ts
web\src\lib\selection.ts
web\src\hooks\useRallySelection.ts
web\src\lib\query.test.ts
web\src\lib\query.ts
web\src\lib\floor.test.ts
web\src\lib\floor.ts
web\src\lib\format.test.ts
web\src\lib\format.ts
web\src\components\QueryBar.examples.test.ts
web\src\test\setup.ts
web\src\hooks\useIngestJob.test.ts
web\src\hooks\useIngestJob.ts
web\src\lib\errors.test.ts
web\src\lib\errors.ts
web\src\lib\errorText.test.ts
web\src\lib\errorText.ts
web\src\api\client.test.ts
web\src\api\client.ts
web\src\api\types.ts

```

```
web\src\api\upload.test.ts
web\src\hooks\useUpload.test.ts
web\src\hooks\useUpload.ts
web\src\vite-env.d.ts
web\src\i18n\index.ts
web\src\i18n\locales\en\common.json
web\src\i18n\locales\kn\common.json
web\src\i18n\locales\hi\common.json
web\src\i18n\locales\es\common.json
web\src\i18n\locales\da\common.json
web\src\i18n\locales\id\common.json
web\src\i18n\hygiene.test.ts
```

## 8. assistant (2026-06-21T17:17:58.606Z)

Now let me read the en catalog tap.\* block, the query.\* block (to compare copy), the hygiene test, the parser, and the tap==type test.

## 9. user (2026-06-21T17:17:59.661Z)

```
1      {
2        "app": {
3          "badge": "pick your rallies · alpha",
4          "title": "Pick the rallies for your reel",
5          "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
6            tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
7        },
8        "lang": {
9          "label": "Language",
10         "en": "English",
11         "kn": "?????",
12         "hi": "??????",
13         "es": "Español",
14         "da": "Dansk",
15         "id": "Bahasa Indonesia"
16       },
17       "bulk": {
18         "selectAll": "Select all",
19         "clear": "Clear",
20         "invert": "Invert"
21       },
22       "mode": {
23         "demo": "These are sample rallies ? process a video above to work on a real match.",
24         "loading": "Finding the rallies in your video?",
25         "error": "Couldn't load this video's rallies: {error}",
26         "loaded": "Found <b>{count}</b> rallies in your video."
27       },
28       "query": {
29         "placeholder": "e.g. over 10s · top 15 longest · shortest first",
30         "ariaLabel": "Filter rallies by length, count or order",
31         "apply": "Apply",
32         "notAvailable": "not available yet",
33         "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
34         "notAvailableHelp": "Shot type, players and score aren't available yet ? filter by
35           rally length instead.",
36         "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
37         "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
38         "try": "Sample queries:",
39         "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
40           <x>experimental</x> (under development)."
41       },
42       "grid": {
43         "empty": "No rallies to show yet.",
44         "rally": "Rally {n}",
45         "inReel": "Rally {n}, in reel",
```

```

43     "notInReel": "Rally {n}, not in reel",
44     "start": "start",
45     "long": "long",
46     "shots": "~{count} shots · experimental",
47     "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
48     "shotsConfidence": " ? confidence: {conf}"
49 },
50     "footer": {
51         "count": "{count, plural, one {# rally} other {# rallies}}",
52         "summaryRest": "selected · {total} total",
53         "reelName": "Reel name",
54         "reelNamePlaceholder": "my-highlights",
55         "build": "Build reel ({count})",
56         "building": "Building?",
57         "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? it would
be skipped, so the build would fail. Pick a longer rally.",
58         "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ?
they'd all be skipped, so the build would fail. Pick longer rallies.",
59         "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
60         "floorSomeRest": "with fewer than {min} shots will be skipped when building (they're
too short).",
61     },
62     "result": {
63         "ready": "Your reel is ready",
64         "failed": "Build failed",
65         "building": "Stitching your reel?",
66         "buildingReassure": "Joining your selected rallies into one video ? this usually
only takes a moment.",
67         "dismiss": "Dismiss",
68         "download": "Download {filename}"
69     },
70     "build": {
71         "noVideo": "These are sample rallies ? process a real video first to build a reel."
72     },
73     "ingest": {
74         "title": "Process a video",
75         "intro": "Point KhelSutra at a video and it finds the rallies for you ? this can
take a few minutes.",
76         "uriPlaceholder": "gs://your-bucket/match.mp4",
77         "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
78         "hint": "Paste a cloud-storage link (gs://?), or a path to a video on the machine
running KhelSutra.",
79         "submit": "Process video",
80         "submitting": "Submitting?",
81         "cancel": "Cancel",
82         "working": "Processing your video?",
83         "progress": "{progress}",
84         "retry": "Try again",
85         "reassure": "Safe to wait ? KhelSutra keeps working even if your connection is slow;
a long video can take several minutes.",
86         "statusFailed": "Couldn't process",
87         "chooseFile": "Choose a video file",
88         "uploadAria": "Choose a video file to upload",
89         "fileHint": "Pick an .mp4 or .mov from this device ? it uploads, then KhelSutra
finds the rallies.",
90         "orLink": "Or use a cloud link",
91         "uploadingStatus": "Uploading?",
92         "uploadProgress": "Upload progress"
93     },
94     "errors": {
95         "networkError": "Can't reach KhelSutra. Is it running?",
96         "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine
running KhelSutra.",
97         "emptyResult": "Processing finished but found no rallies in this video. Try a

```

```

clearer match recording.",
98     "timeout": "This is taking longer than expected. The job may still be running ?
reload with the video link, or try again.",
99     "engineGeneric": "Something went wrong. Please try again."
100   },
101   "tap": {
102     "title": "Or tap to filter:",
103     "longest": "Longest first",
104     "shortest": "Shortest first",
105     "over10": "Over 10 seconds",
106     "over20": "Over 20 seconds",
107     "top5": "Top 5 longest",
108     "top10": "Top 10 longest"
109   },
110   "selection": {
111     "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
112   }
113 }
114

```

#### 10. user (2026-06-21T17:17:59.793Z)

```

1    // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2    //   1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3    //     drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4    //     starts enforcing the moment a kn/hi catalog lands (B5/B6).
5    //   2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6    //     aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7    //     the doc; an AST-level sweep of JSX text can extend it later.
8    import { describe, it, expect } from "vitest";
9    import { readdirSync, readFileSync } from "node:fs";
10   import { join } from "node:path";
11
12   const LOCALES_DIR = join(import.meta.dirname, "locales");
13   const SRC_DIR = join(import.meta.dirname, "..");
14
15   function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16     const keys: string[] = [];
17     for (const [k, v] of Object.entries(obj)) {
18       const key = prefix ? `${prefix}.${k}` : k;
19       if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20       else keys.push(key);
21     }
22     return keys.sort();
23   }
24
25   const loadKeys = (locale: string): string[] =>
26     flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28   describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29     const enKeys = loadKeys("en");
30     const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31       .filter((d) => d.isDirectory() && d.name !== "en")
32       .map((d) => d.name);
33
34     it("en is the non-empty source catalog", () => {
35       expect(enKeys.length).toBeGreaterThan(20);
36     });
37

```

```

38     for (const loc of others) {
39         it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40             const keys = loadKeys(loc);
41             const missing = enKeys.filter((k) => !keys.includes(k));
42             const extra = keys.filter((k) => !enKeys.includes(k));
43             expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });});
44         });
45     }
46 });
47
48 // Key-parity proves every locale has the same KEYS; this proves every locale's VALUE
keeps the same
49 // simple `{placeholder}` set as en ? so a translation can never silently drop a
{count}/{error}/{min}/
50 // {filename}/{n}/? and break interpolation. (ICU plural headers like `{count, plural,
?}` carry no
51 // simple {word} placeholder and match as empty on both sides, so they're correctly
ignored.)
52 describe("placeholder parity ? every locale preserves en's {placeholders}", () => {
53     function flatEntries(obj: Record<string, unknown>, prefix = ""): Record<string,
string> {
54         const out: Record<string, string> = {};
55         for (const [k, v] of Object.entries(obj)) {
56             const key = prefix ? `${prefix}.${k}` : k;
57             if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
58             else out[key] = String(v);
59         }
60         return out;
61     }
62     const placeholders = (s: string) => (s.match(/\{(\w+)\}/g) ?? []).sort();
63     const load = (loc: string) => flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR,
loc, "common.json"), "utf8")));
64     const enE = load("en");
65     const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
66         .filter((d) => d.isDirectory() && d.name !== "en")
67         .map((d) => d.name);
68
69     for (const loc of others) {
70         it(`${loc} preserves every {placeholder} that en uses`, () => {
71             const locE = load(loc);
72             const mismatches: Array<{ key: string; en: string[]; got: string[] }> = [];
73             for (const key of Object.keys(enE)) {
74                 if (!(key in locE)) continue; // missing keys are the key-parity test's job
75                 const a = placeholders(enE[key]);
76                 const b = placeholders(locE[key]);
77                 if (a.join(",") !== b.join(",")) mismatches.push({ key, en: a, got: b });
78             }
79             expect(mismatches).toEqual([]);
80         });
81     }
82 });
83
84 describe("es 3-form plural (CLDR one/many/other) ? MULTILINGUAL Phase A", () => {
85     // Spanish cardinal is {one, many, other}; every es ICU plural message MUST carry a
`many` arm
86     // (the `many` category fires on large/compact numbers) or intl-messageformat silently
falls back.
87     // These are the plural-bearing keys in the catalog.
88     const es = JSON.parse(readFileSync(join(LOCALES_DIR, "es", "common.json"), "utf8")) as
Record<string, Record<string, string>>;
89     const pluralKeys: Array<[string, string]> = [
90         ["footer", "count"],
91         ["footer", "floorSomeLead"],

```

```

92     ["selection", "count"],
93 ];
94 for (const [ns, key] of pluralKeys) {
95     it(`es ${ns}.${key} carries one/many/other arms`, () => {
96         const msg = es[ns][key];
97         expect(msg, `${ns}.${key}`).toMatch(/\\bone\\s*\\/);
98         expect(msg, `${ns}.${key}`).toMatch(/\\bmany\\s*\\/);
99         expect(msg, `${ns}.${key}`).toMatch(/\\bother\\s*\\/);
100     });
101 }
102 });
103
104 describe("no-bare-strings ? user-facing component attributes go through t()", () => {
105     const FILES = [
106         "App.tsx",
107         "components/QueryBar.tsx",
108         "components/TapFilters.tsx",
109         "components/RallyGrid.tsx",
110         "components/SelectionFooter.tsx",
111         "components/CompileResult.tsx",
112         "components/LanguageSwitcher.tsx",
113         "components/IngestPanel.tsx",
114     ];
115     // A hardcoded placeholder / aria-label / title with letters is a bare-string
116     regression ? it
117     // must be an expression ({t(...)}), not a quoted literal.
118     const BARE_ATTR = /\\b(?:placeholder|aria-label|title)\\s*=\\s*"\\^[^"]*"\\/g;
119
120     for (const f of FILES) {
121         it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
122             const src = readFileSync(join(SRC_DIR, f), "utf8");
123             expect(src.match(BARE_ATTR) ?? []).toEqual([]);
124         });
125     }
126
127     describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
128         it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google Fonts)", () => {
129             const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
130             expect(main).toMatch(/@fontsource-variable\\/noto-sans-kannada\\/);
131             expect(main).toMatch(/@fontsource-variable\\/noto-sans-devanagari\\/);
132         });
133         it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
134             const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
135             expect(css).toMatch(/Noto Sans Kannada Variable\\/);
136             expect(css).toMatch(/Noto Sans Devanagari Variable\\/);
137         });
138     });
139
140     // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
141     // internet) ?
142     // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
143     // CDN/Google Fonts.
144     describe("offline / self-contained ? no external runtime assets", () => {
145         const CDN = /googleapis|gstatic|fonts\\.google|cdnjs|unpkg|jsdelivr|\\bcdn\\.\\[a-z]\\/i;
146         const FILES = [
147             "../index.html", // web/index.html
148             "main.tsx",
149             "index.css",
150         ];
151         for (const rel of FILES) {
152             it(`${rel}: no external font/CDN reference`, () => {
153                 expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
154             });
155         }
156     });

```

```

153     }
154     it("web/index.html loads no absolute-URL <link>/<script src> (everything is
bundled/local)", () => {
155         const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
156         expect(html).not.toMatch(/(?:href|src)\\s*=\\s*["']https?:\\/\\/i);
157     });
158 });
159

```

## 11. assistant (2026-06-21T17:18:04.313Z)

Now let me look at the parser (query.ts) to verify the tap parts deep-equal what the parser produces, the tap==type test, and the QueryBar examples test.

## 12. user (2026-06-21T17:18:05.425Z)

```

1      // Deterministic, dependency-free query compiler for the rally picker
(PER_RALLY_SELECTION.md
2      // D2/D5). NO LLM, NO network ? a tiny regex grammar over the ONLY honest, queryable
axes the
3      // engine returns today: rally LENGTH (duration), ORDER (start_time / duration /
shot_count),
4      // COUNT (top/first N), and SHOT COUNT (metadata.shot_count).
5      //
6      // Everything else a user might type ? shot TYPE (smash/clear/drop?), PLAYER, SCORE ?
has no
7      // detector and no column, so we FLAG it as "not available yet" and NEVER fabricate a
result.
8      // Parsing is pure and unit-tested; the QueryBar React component is a thin shell over
it.
9      import type { RallySegment } from "../api/types";
10
11     export type CompareOp = ">" | ">=" | "<" | "<=" | "=";
12     export type FilterField = "duration" | "shot_count";
13     /** One ANDed predicate over a rally field (e.g. duration > 10s). */
14     export interface Filter {
15         field: FilterField;
16         op: CompareOp;
17         value: number; // seconds for duration; a raw count for shot_count
18     }
19
20     export type SortKey = "duration" | "start_time" | "shot_count";
21     export interface Sort {
22         key: SortKey;
23         dir: "asc" | "desc";
24     }
25
26     /** A recognized-but-unsupported concept ? surfaced as a greyed "not available yet"
hint. */
27     export type UnavailableKind = "shot-type" | "player" | "score";
28     export interface Unavailable {
29         token: string;
30         kind: UnavailableKind;
31     }
32
33     export interface ParsedQuery {
34         raw: string;
35         /** ANDed together. */
36         filters: Filter[];
37         sort?: Sort;
38         /** "top N" / "first N" / "N longest". */
39         limit?: number;
40         /** Concepts we understood but cannot honour (shot-type / player / score). */
41         unavailable: Unavailable[];
42         /** True when at least one actionable clause (filter | sort | limit) was understood.

```

```

43     recognized: boolean;
44 }
45
46 export interface QueryResult {
47     /** ALL rallies in display order (sorted if a sort was given, else input order). */
48     ordered: RallySegment[];
49     /** Ids matching the filters (then limited), in display order. */
50     selectedIds: number[];
51 }
52
53 // ?? grammar fragments ?????????????????????????????????????????????????????????????????
54 // `(!\d)` stops a greedy digit-run from backtracking and leaving stray trailing digits
55 // (so "over 10 smashes" can't degrade to "over 1" + "0 smashes").
56 const NUM = "(\\d+(?:\\.\\d+)?)(!\\d)";
57 // Time / "shots" unit tokens. "shots?" is FIRST so it isn't eaten by the bare "s";
58 "mins?"
59 // already covers "min" (no redundant bare "min"); \b stops "m" matching "minutes" etc.
60 const UNIT_INNER = "shots?|seconds?|secs?|minutes?|mins?|m|s";
61 const UNIT = `(?:\\s*(${UNIT_INNER})\\b)?`;
62 // Shot-TYPE / player / score nouns ? concepts with NO detector and NO column. A number
63 trailing
64 // one of these must NEVER become a real filter (honesty: never reroute an unsupported
65 concept
66 // onto the duration axis ? PER_RALLY_SELECTION.md §3/§4.3). These stems are shared with
67 the
68 // "not available yet" flagging below so the two can never drift apart.
69 const SHOT_TYPE = "smash(?:es)?|clears?|drops?|dropshots?|net
70 ?shots?|drives?|lifts?|serves?|kills?|taps?|pushes?|blocks?|slices?";
71 const PLAYER = "players?|opponents?|servers?|receivers?";
72 const SCORE = "points?|score|deuce|game point|set point|match point";
73 const UNAVAIL_WORD = `(?:${SHOT_TYPE}|${PLAYER}|${SCORE})`;
74 // Voids a comparator clause whose number is immediately followed by an unsupported
75 noun.
76 const NOT_UNAVAIL = `(?!\\s*${UNAVAIL_WORD}\\b)`;
77
78 interface OpPattern {
79     op: CompareOp;
80     re: RegExp;
81     /** Which capture group holds the number / the unit. */
82     numAt: number;
83     unitAt: number;
84 }
85
86 // Leading comparator ("over 10s"). The "no more than" / "at least" guards come BEFORE
87 their
88 // bare counterparts ("more than" / "less than") so the negated phrase is consumed
89 first.
90 function lead(alt: string, op: CompareOp): OpPattern {
91     return { op, re: new RegExp(`(?:${alt})\\s*${NUM}${UNIT}${NOT_UNAVAIL}`, "g"), numAt:
92 1, unitAt: 2 };
93 }
94
95 const LEADING: OpPattern[] = [
96     lead("no more than|at most|maximum(?: of)?|max|up to|<=", "<="),
97     lead("no less than|no fewer than|at least|minimum(?: of)?|min|>=", ">="),
98     lead("exactly|equal to|equals", "="),
99     lead("over|longer than|more than|greater than|above|bigger than|>", ">"),
100     lead("under|shorter than|less than|fewer than|below|smaller than|<", "<"),
101 ];
102
103 // Trailing comparator. The unit may sit BEFORE the operator ("10s+", "8 shots or more")
104 or the
105 // "shots" noun AFTER it ("10+ shots") ? capture both sides and use whichever is
106 present.

```

```

96     // NOT_UNAVAIL guards the same way ("5+ smashes" must not become duration ? 5).
97     interface TrailPattern {
98         op: CompareOp;
99         re: RegExp;
100     }
101     const POST_NOUN = "(?:\\s*(shots?)\\b)?"
102     const TRAILING: TrailPattern[] = [
103         { op: ">=", re: new RegExp(`${NUM}${UNIT}\\s*(?:\\+|or
104 (?:more|longer|over|above|greater))`${POST_NOUN}${NOT_UNAVAIL}`, "g") },
105         { op: "<=", re: new RegExp(`${NUM}${UNIT}\\s*or
106 (?:less|fewer|shorter|under|below)``${POST_NOUN}${NOT_UNAVAIL}`, "g") },
107     ];
108
109     function toSeconds(n: number, unit?: string): number {
110         return unit && /\m/.test(unit) ? n * 60 : n; // m/min/minute(s) ? seconds; else as-is
111     }
112
113     function makeFilter(numStr: string, unit: string | undefined, op: CompareOp): Filter {
114         const n = Number(numStr);
115         const isShots = !!unit && /\shots?$/i.test(unit);
116         return isShots
117             ? { field: "shot_count", op, value: n }
118             : { field: "duration", op, value: toSeconds(n, unit) };
119     }
120
121     const SORT_PATTERNS: Array<RegExp, Sort> = [
122         [/b(?:longest(?: first)?|longer first|by (?:duration|length)|duration desc)\b/, {
123 key: "duration", dir: "desc" }],
124         [/b(?:shortest(?: first)?|shorter first|duration asc)\b/, { key: "duration", dir:
125 "asc" }],
126         [/b(?:newest(?: first)?|latest|most recent|recent first)\b/, { key: "start_time",
127 dir: "desc" }],
128         [/b(?:oldest(?: first)?|earliest|chronological|in order|by time)\b/, { key:
129 "start_time", dir: "asc" }],
130         [/b(?:most shots(?: first)?|highest shots?|by shots?|by shot count|shot count
131 desc)\b/, { key: "shot_count", dir: "desc" }],
132         [/b(?:fewest shots(?: first)?|least shots?|lowest shots?|shot count asc)\b/, { key:
133 "shot_count", dir: "asc" }],
134     ];
135
136     // Order before kind so a token claimed by an earlier kind isn't double-counted. The
137     // stems are the
138     // same SHOT_TYPE/PLAYER/SCORE used by NOT_UNAVAIL, so "flag it" and "don't filter on
139     // it" stay in sync.
140     const UNAVAILABLE_PATTERNS: Array<UnavailableKind, RegExp> = [
141         ["shot-type", new RegExp(`\\b(${SHOT_TYPE})\\b`, "g")],
142         ["player", new RegExp(`\\b(${PLAYER})\\b`, "g")],
143         ["score", new RegExp(`\\b(${SCORE})\\b`, "g")],
144     ];
145
146     // A magnitude that carries a time/"shots" unit is NOT an ordinal count ("first 30
147     // seconds" ? the
148     // first 30 rallies). This blocks the limit regexes from misreading such a number as a
149     // count.
150     const NOT_UNIT = `(?!\\s*(?:${UNIT_INNER})\\b)`
151
152     // ?? public API ?????????????????????????????????????????????????????????????
153
154     export function parseQuery(raw: string): ParsedQuery {
155         const normalized = raw
156             .toLowerCase()
157             .replace(/>/g, ">=")
158             .replace(/</g, "<=")
159             .replace(/(\d),(?=\d{3}\b)/g, "$1") // strip thousands separators ("1,000 shots" ?
160 "1000 shots")

```

```

148     .replace(/,/g, " ") // any remaining comma is a clause separator
149     .replace(/\s+/g, " ")
150     .trim();
151
152     const filters: Filter[] = [];
153     let working = normalized;
154
155     // Extract comparator clauses, blanking each match so later passes can't re-read its
number.
156     for (const pat of LEADING) {
157         working = working.replace(pat.re, (full, ...g) => {
158             filters.push(makeFilter(g[pat.numAt - 1], g[pat.unitAt - 1], pat.op));
159             return " ".repeat(full.length);
160         });
161     }
162     for (const pat of TRAILING) {
163         working = working.replace(pat.re, (full, num, preUnit, postNoun) => {
164             filters.push(makeFilter(num, preUnit ?? postNoun, pat.op));
165             return " ".repeat(full.length);
166         });
167     }
168
169     // Explicit sort (runs before limit so "top 15 longest" keeps the longest?desc
intent).
170     let sort: Sort | undefined;
171     for (const [re, s] of SORT_PATTERNS) {
172         if (re.test(working)) {
173             sort = s;
174             break;
175         }
176     }
177
178     // Limit ("top/best N", "first N", "last N", "N longest/shortest", "longest/shortest
N"). A number
179     // carrying a unit (\b + NOT_UNIT) is NOT a count, so "first 30 seconds" / "longest 3
shots" don't
180     // become counts. `sortFromLimit` tracks a sort implied SOLELY by the limit, so if the
limit turns
181     // out invalid (e.g. "top 0") we drop that phantom sort too and the whole query reads
as a no-op.
182     let limit: number | undefined;
183     let sortFromLimit = false;
184     let m: RegExpMatchArray | null;
185     if (
186         (m = working.match(new RegExp(`\\b(\\d+)\\b${NOT_UNIT}\\s+(longest|shortest)\\b`)))
||
187         (m = working.match(new RegExp(`\\b(longest|shortest)\\s+(\\d+)\\b${NOT_UNIT}`)))
188     ) {
189         const num = /^\\d/.test(m[1]) ? m[1] : m[2];
190         const word = /^\\d/.test(m[1]) ? m[2] : m[1];
191         limit = parseInt(num, 10);
192         if (!sort) {
193             sort = { key: "duration", dir: word === "longest" ? "desc" : "asc" };
194             sortFromLimit = true;
195         }
196     } else if ((m = working.match(new RegExp(`\\b(?:top|best)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
197         limit = parseInt(m[1], 10);
198         if (!sort) {
199             sort = { key: "duration", dir: "desc" }; // "top N" ? best = longest
200             sortFromLimit = true;
201         }
202     } else if ((m = working.match(new
RegExp(`\\b(?:first|earliest)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
203         limit = parseInt(m[1], 10);

```

```

204     if (!sort) {
205         sort = { key: "start_time", dir: "asc" };
206         sortFromLimit = true;
207     }
208     } else if ((m = working.match(new RegExp(`\\b(?:last|latest|newest|most
recent)\\s+(\\d+)\\b${NOT_UNIT}`)))) {
209         limit = parseInt(m[1], 10);
210         if (!sort) {
211             sort = { key: "start_time", dir: "desc" };
212             sortFromLimit = true;
213         }
214     }
215
216     // A limit must be a positive integer. Reject "top 0" (which would otherwise slice to
an EMPTY
217     // selection and silently wipe the user's curation) and clamp absurd magnitudes so the
pill never
218     // renders scientific notation.
219     if (limit != null) {
220         if (!Number.isInteger(limit) || limit < 1) {
221             limit = undefined;
222             if (sortFromLimit) sort = undefined;
223         } else if (limit > Number.MAX_SAFE_INTEGER) {
224             limit = Number.MAX_SAFE_INTEGER;
225         }
226     }
227
228     // Unavailable concepts (scanned on the normalized text; deduped, first kind wins).
229     const unavailable: Unavailable[] = [];
230     const seen = new Set<string>();
231     for (const [kind, re] of UNAVAILABLE_PATTERNS) {
232         for (const mm of normalized.matchAll(re)) {
233             const token = mm[1];
234             if (!seen.has(token)) {
235                 seen.add(token);
236                 unavailable.push({ token, kind });
237             }
238         }
239     }
240
241     return {
242         raw,
243         filters,
244         sort,
245         limit,
246         unavailable,
247         recognized: filters.length > 0 || sort != null || limit != null,
248     };
249 }
250
251 function rallyValue(r: RallySegment, key: SortKey): number | undefined {
252     if (key === "duration") return r.duration;
253     if (key === "start_time") return r.start_time;
254     return r.metadata?.shot_count;
255 }
256
257 /** Stable sort; rallies missing the sort field always sort LAST (unknown ? smallest).
*/
258 export function sortRallies(rallies: RallySegment[], sort: Sort): RallySegment[] {
259     return rallies
260         .map((r, i) => ({ r, i }))
261         .sort((a, b) => {
262             const av = rallyValue(a.r, sort.key);
263             const bv = rallyValue(b.r, sort.key);
264             if (av == null && bv == null) return a.i - b.i;

```

```

265         if (av == null) return 1;
266         if (bv == null) return -1;
267         if (av !== bv) return sort.dir === "asc" ? av - bv : bv - av;
268         return a.i - b.i; // tie ? preserve input order
269     })
270     .map((x) => x.r);
271 }
272
273 function matchesFilter(r: RallySegment, f: Filter): boolean {
274     const v = f.field === "duration" ? r.duration : r.metadata?.shot_count;
275     if (v == null) return false; // a rally without shot_count can't satisfy a shot filter
276     switch (f.op) {
277         case ">":
278             return v > f.value;
279         case ">=":
280             return v >= f.value;
281         case "<":
282             return v < f.value;
283         case "<=":
284             return v <= f.value;
285         case "=":
286             return v === f.value;
287         default:
288             return false;
289     }
290 }
291
292 /** Apply a parsed query to the rallies: ordered display list + the ids it selects.
Pure.
293     * An UNRECOGNIZED query (nothing understood) selects nothing and leaves order untouched
?
294     * so garbage input can never silently wipe a user's manual curation. The App also
guards on
295     * `q.recognized` before applying, so an empty box is a no-op end to end. */
296 export function runQuery(rallies: RallySegment[], q: ParsedQuery): QueryResult {
297     if (!q.recognized) return { ordered: rallies.slice(), selectedIds: [] };
298     const ordered = q.sort ? sortRallies(rallies, q.sort) : rallies.slice();
299     let matched = ordered.filter((r) => q.filters.every((f) => matchesFilter(r, f)));
300     if (q.limit !== null) matched = matched.slice(0, q.limit);
301     return { ordered, selectedIds: matched.map((r) => r.id) };
302 }
303
304 // ?? human-readable labels (shared by the QueryBar pill and the tests) ??????????
305 const OP_SYMBOL: Record<CompareOp, string> = { ">": ">", ">=": "?", "<": "<", "<=": "?",
"=": "=" };
306
307 export function filterLabel(f: Filter): string {
308     if (f.field === "duration") return `length ${OP_SYMBOL[f.op]} ${f.value}s`;
309     return `shots ${OP_SYMBOL[f.op]} ${f.value}`;
310 }
311
312 export function sortLabel(s: Sort): string {
313     if (s.key === "duration") return s.dir === "desc" ? "longest first" : "shortest
first";
314     if (s.key === "start_time") return s.dir === "desc" ? "newest first" : "oldest first";
315     return s.dir === "desc" ? "most shots first" : "fewest shots first";
316 }
317
318 /** The count label, derived from the limit's actual sort so the pill can't contradict
itself
319     * ("first 10", not "top 10", when the sort is oldest-first). */
320 export function limitLabel(limit: number, sort?: Sort): string {
321     if (sort?.key === "start_time") return sort.dir === "asc" ? `first ${limit}` : `last
${limit}`;
322     if (sort?.key === "duration" && sort.dir === "asc") return `shortest ${limit}`;

```

```

323     if (sort?.key === "shot_count") return sort.dir === "asc" ? `fewest ${limit}` : `top
${limit} by shots`;
324     return `top ${limit}`; // duration desc, or no sort
325   }
326
327   /** Human clause labels for the parsed-pill (e.g. ["length > 10s", "longest first", "top
15"]). */
328   export function summarize(q: ParsedQuery): string[] {
329     const parts: string[] = [];
330     for (const f of q.filters) parts.push(filterLabel(f));
331     if (q.sort) parts.push(sortLabel(q.sort));
332     if (q.limit !== null) parts.push(limitLabel(q.limit, q.sort));
333     return parts;
334   }
335

```

### 13. user (2026-06-21T17:18:05.754Z)

```

1     import { describe, it, expect } from "vitest";
2     import { EXAMPLES } from "../QueryBar";
3     import { parseQuery } from "../lib/query";
4
5     // The example chips apply on tap, so a broken example would silently do nothing (or
show a
6     // "couldn't read that"). Pin that EVERY shipped example is a recognized, fully-
available query
7     // (no unsupported tokens) ? a regression guard for the duration-led example set.
8     describe("QueryBar examples", () => {
9       it("are all recognized, actionable, and fully available (no experimental/unsupported
tokens)", () => {
10         for (const ex of EXAMPLES) {
11           const q = parseQuery(ex);
12           expect(q.recognized, ex).toBe(true);
13           expect(q.unavailable, ex).toEqual([]);
14           // every example must actually constrain or order the set
15           expect(q.filters.length > 0 || q.sort !== null || q.limit !== null, ex).toBe(true);
16         }
17       });
18
19       it("lead with rally length, not the experimental shot-count axis", () => {
20         const shotExamples = EXAMPLES.filter((ex) => parseQuery(ex).filters.some((f) =>
f.field === "shot_count"));
21         expect(shotExamples).toEqual([]);
22       });
23     });
24

```

### 14. assistant (2026-06-21T17:18:09.345Z)

Now let me find the tap==type test that pins parseQuery(f.english) deep-equals f.parts.

### 15. user (2026-06-21T17:18:10.748Z)

Found 3 files  
web\src\components\TapFilters.test.tsx  
web\src\App.tsx  
web\src\components\TapFilters.tsx

### 16. user (2026-06-21T17:18:14.826Z)

```

1     // @vitest-environment jsdom
2     //
3     // A9 ? localized tap-only filters. The load-bearing guarantee: each tap selects
IDENTICALLY to

```

```

4      // typing its English equivalent (tap == type), so a Kannada/Hindi/? user who taps gets
exactly what
5      // an English user who types gets ? the parser stays the single source of truth.
6      import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
7      import { render, screen, cleanup, fireEvent } from "@testing-library/react";
8      import i18n from "../i18n";
9      import { TapFilters, TAP_FILTERS } from "../TapFilters";
10     import { parseQuery } from "../lib/query";
11
12     beforeEach(async () => {
13       await i18n.changeLanguage("en");
14     });
15     afterEach(() => cleanup());
16
17     describe("TapFilters (A9)", () => {
18       it("each tap builds the SAME query as typing its English equivalent (tap == type)", ()
=> {
19         for (const f of TAP_FILTERS) {
20           const typed = parseQuery(f.english);
21           expect(typed.recognized, f.english).toBe(true);
22           expect(typed.unavailable, f.english).toEqual([]); // length-led, fully available ?
never shot-count
23           expect({ filters: typed.filters, sort: typed.sort, limit: typed.limit },
f.english).toEqual({
24             filters: f.parts.filters,
25             sort: f.parts.sort,
26             limit: f.parts.limit,
27           });
28         }
29       });
30
31       it("tapping a control applies a recognized query through onApply", () => {
32         const onApply = vi.fn();
33         render(<TapFilters onApply={onApply} />);
34         fireEvent.click(screen.getByRole("button", { name: "Over 10 seconds" }));
35         expect(onApply).toHaveBeenCalledTimes(1);
36         const [q, source] = onApply.mock.calls[0];
37         expect(source).toBe("example");
38         expect(q.recognized).toBe(true);
39         expect(q.filters).toEqual([ { field: "duration", op: ">", value: 10 } ]);
40         expect(q.unavailable).toEqual([]);
41       });
42
43       it("renders localized control labels (Kannada)", async () => {
44         await i18n.changeLanguage("kn");
45         render(<TapFilters onApply={() => {}} />);
46         expect(screen.getByText("??? ?????? ??????")).toBeTruthy(); // tap.longest ? Kannada
47         await i18n.changeLanguage("en");
48       });
49
50       it("every control is a ?44px tap target (A4)", () => {
51         render(<TapFilters onApply={() => {}} />);
52         for (const f of TAP_FILTERS) {
53           expect(screen.getByText(i18n.t(f.key)).className, f.key).toMatch(/min-h-11/);
54         }
55       });
56     });
57

```

## 17. assistant (2026-06-21T17:18:18.657Z)

Now let me check the localized tap.\* blocks across the other catalogs to verify the labels are honest and to spot copy issues. Let me read the tap and query blocks from kn, hi, es, da, id.

## 18. user (2026-06-21T17:18:22.226Z)

```

=== kn ===
{
  "title": "???? ?????? ????? ?????? ????:",
  "longest": "??? ?????? ??????",
  "shortest": "??? ?????? ??????",
  "over10": "10 ????????????? ??????",
  "over20": "20 ????????????? ??????",
  "top5": "??? ????? 5",
  "top10": "??? ????? 10"
}
--- query.try / placeholder ---
try: ????? ??????????:
hint: 10 ????????????? ?????? ?????????????? over 10s ????? ????? ? ????? ?????? ??????????
?????? ??????.
=== hi ===
{
  "title": "? ? ?????? ????? ?? ??? ?? ?????:",
  "longest": "????? ??? ????",
  "shortest": "????? ??? ????",
  "over10": "10 ????? ? ? ??????",
  "over20": "20 ????? ? ? ??????",
  "top5": "????? ??? ???? 5",
  "top10": "????? ??? ???? 10"
}
--- query.try / placeholder ---
try: ????? ??????:
hint: 10 ????? ? ???? ??????? ? ???? over 10s ????? ????? ? ? ???? ? ? ???? ???? ??????
=== es ===
{
  "title": "0 toca para filtrar:",
  "longest": "Las más largas primero",
  "shortest": "Las más cortas primero",
  "over10": "Más de 10 segundos",
  "over20": "Más de 20 segundos",
  "top5": "Las 5 más largas",
  "top10": "Las 10 más largas"
}
--- query.try / placeholder ---
try: Consultas de ejemplo:
hint: Escribe over 10s para jugadas de más de 10 segundos ? o toca un ejemplo abajo.
=== da ===
{
  "title": "Eller tryk for at filtrere:",
  "longest": "Længste først",
  "shortest": "Korteste først",
  "over10": "Over 10 sekunder",
  "over20": "Over 20 sekunder",
  "top5": "De 5 længste",
  "top10": "De 10 længste"
}
--- query.try / placeholder ---
try: Eksempelforespørgsler:
hint: Skriv over 10s for dueller længere end 10 sekunder ? eller tryk på et eksempel nedenfor.
=== id ===
{
  "title": "Atau ketuk untuk memfilter:",
  "longest": "Terpanjang dulu",
  "shortest": "Terpendek dulu",
  "over10": "Lebih dari 10 detik",
  "over20": "Lebih dari 20 detik",
  "top5": "5 terpanjang",
  "top10": "10 terpanjang"
}
--- query.try / placeholder ---
try: Contoh kueri:

```

hint: Ketik over 10s untuk reli lebih dari 10 detik ? atau ketuk contoh di bawah.

### 19. assistant (2026-06-21T17:18:27.532Z)

Now I have a full picture. Let me run the web test suite to verify the claim of 173 green tests, and confirm the tap==type relationship holds (particularly checking the deep-equal logic for the `sort`/`limit` being `undefined` in some parts).

### 20. user (2026-06-21T17:18:31.561Z)

ok

### 21. user (2026-06-21T17:18:48.864Z)

```
stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T17:18:47.713Z","level":"error","event":"api.network_error","cid":"cb14464e-6f37-4ac9-bc79-0464a3286e20","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stderr | src/i18n/i18n.test.tsx > browser-language negotiation (B1:
N1/N3?N7) > N6: an unsupported browser language (Tamil) serves English content ?
never a locale we can't render
{"ts":"2026-06-21T17:18:47.714Z","level":"warn","event":"config.unavailable","error":"ApiError: Network error
calling GET /api/config"}

stdout | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:47.759Z","level":"info","event":"api.request","cid":"ae809ed8-4b2f-4c7c-a874-488e16cfffbd","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A
(es/da/id) > renders each new locale's catalog (real translations, not English /
not raw keys)
{"ts":"2026-06-21T17:18:47.800Z","level":"error","event":"api.network_error","cid":"ae809ed8-4b2f-4c7c-a874-488e16cfffbd","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stdout | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:47.842Z","level":"info","event":"api.request","cid":"d95aa7d1-0c07-4e9e-be48-659e9e8506c4","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A
(es/da/id) > renders each new locale's catalog (real translations, not English /
not raw keys)
{"ts":"2026-06-21T17:18:47.880Z","level":"error","event":"api.network_error","cid":"d95aa7d1-0c07-4e9e-be48-659e9e8506c4","method":"GET","url":"/api/config","error":"Error: no engine in test"}

stdout | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A (es/da/id) >
renders each new locale's catalog (real translations, not English / not raw keys)
{"ts":"2026-06-21T17:18:47.923Z","level":"info","event":"api.request","cid":"3a1cd324-e578-477d-acd1-88302a7dc926","method":"GET","url":"/api/config"}

stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A
(es/da/id) > renders each new locale's catalog (real translations, not English /
not raw keys)
{"ts":"2026-06-21T17:18:47.971Z","level":"error","event":"api.network_error","cid":"3a1cd324-e578-477d-acd1-88302a7dc926","method":"GET","url":"/api/config","error":"Error: no engine in test"}
{"ts":"2026-06-
```

```
21T17:18:47.971Z", "level": "warn", "event": "config.unavailable", "error": "ApiError: Network error calling GET /api/config"}
```

```
stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A
(es/da/id) > renders each new locale's catalog (real translations, not English /
not raw keys)
{"ts": "2026-06-
21T17:18:47.971Z", "level": "warn", "event": "config.unavailable", "error": "ApiError: Network error calling GET /api/config"}
```

```
stderr | src/i18n/i18n.test.tsx > MULTILINGUAL Phase A
(es/da/id) > renders each new locale's catalog (real translations, not English /
not raw keys)
{"ts": "2026-06-
21T17:18:47.971Z", "level": "warn", "event": "config.unavailable", "error": "ApiError: Network error calling GET /api/config"}
```

```
? src/i18n/i18n.test.tsx (17 tests)
2157ms
? i18n scaffold (B1) > renders user-facing copy from the en
catalog (not raw keys) 936ms
```

```
Test Files 19 passed (19)
Tests 173 passed (173)
Start at 22:48:39
Duration 8.07s (transform 2.81s, setup 641ms, collect 16.80s, tests 8.87s,
environment 32.08s, prepare 5.37s)
```

## 22. assistant (2026-06-21T17:18:55.263Z)

173 tests pass. Now let me verify tsc + build are clean, and confirm the tap==type deep-equal subtlety (the ``over10`/`over20`` parts omit ``sort`/`limit`` keys, while ``longest`/`shortest`` omit ``limit``).

## 23. user (2026-06-21T17:19:02.893Z)

```
TSC_EXIT=0
```

## 24. assistant (2026-06-21T17:19:07.967Z)

TSC clean. Now let me verify a subtle a11y/UX point: whether the tap.over10 chips, when applied, give an ``unavailable: []`` and whether there's a discoverability/consistency concern. Let me also check the i18n index to confirm how missing keys behave and whether there's a ``data-testid`` concern. Let me also examine the FRIENDLY\_AND\_MULTILINGUAL tracker doc §6.5/§7 to assess faithfulness.

## 25. user (2026-06-21T17:19:09.254Z)

```
/c/Users/avidu/Projects/khelsutra/.git/logs/refs/remotes/origin/docs/friendly-multilingual-
project
/c/Users/avidu/Projects/khelsutra/.git/logs/refs/remotes/origin/docs/friendly-multilingual-spa-
done
/c/Users/avidu/Projects/khelsutra/.git/refs/remotes/origin/docs/friendly-multilingual-project
/c/Users/avidu/Projects/khelsutra/.git/refs/remotes/origin/docs/friendly-multilingual-spa-done
/c/Users/avidu/Projects/khelsutra/docs/FRIENDLY_AND_MULTILINGUAL.md
```

## 26. user (2026-06-21T17:19:13.817Z)

```
2-
3-> **Status:** `DRAFT` ? owner-gated (implementation starts only on explicit approval) .
**Owner:** Avi Dullu . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4-> **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE).
5:> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; thread in `NEXT_STEPS.md`; pointer in engine `docs/DOC_STATUS.md`; memory
```

```

`current-state` + `next-session-ux-localization`.
6-> **Relation to existing docs:** this is the **implementation project** for two engine design
docs ? it *adapts & supersedes-in-practice* `badminton-highlight-indexer/docs/I18N_PLAN.md`
(localization) and the UX half of `~/docs/UI_PROPOSAL.md` (non-tech-savvy console). Those stay
the canonical *design*; this doc carries the *tracked execution* against today's surfaces.
7-> **Honesty note:** each claim tagged `[verified]` (checked against code this session),
`[researched]` (needs sign-off), or `[design]` (proposed). Not legal advice ? the OFL font
question (D-FONT) is owner/counsel-gated.
8-
--
21-## 2. Decisions locked (structural ? owner directive 2026-06-21)
22-| # | Decision | Source / date | Implication |
23-|---|-----|-----|-----|
24-| D1 | **Isolate as ONE tracked project** spanning both workstreams (A: UX, B: kn/hi l10n) |
owner, 2026-06-21 | This doc; §7 = source of truth; archived on DoD |
25-| D2 | **Home = `khelsutra/docs/`** (the implementation repo) | owner-gated tracked-doc
convention | `web/` + `site/` both live here; §7 tracks khelsutra PRs; engine P2 backend-l10n is
a cross-repo deferred row |
26-| D3 | **Adapt `I18N_PLAN.md` to today's surfaces:** P1 (vanilla `t()` shim) ? `site/`; P3
(react-i18next) ? `web/`; **drop the dead `backend/api/static` retrofit** | `[verified]` the doc
targets `backend/api/static` (`I18N_PLAN.md:25,30,156,191`) which exists only in the engine
repo, not here | The doc's framework-agnostic *foundation* (catalog format + negotiation + ICU
plurals + CI) survives unchanged; only the wiring re-points |
27-| D4 | **Engine P2 backend-localization stays owner-gated + cross-repo** (not in this DoD) |
`CLAUDE.md` guardrails | Dynamic engine error/report text localizes later; tracked as a deferred
row |
28-| D5 | **Archive trigger = en+kn+hi live on BOTH surfaces** (+ the non-tech UX baseline),
then run the engine `DOC_STATUS.md §2` archival checklist | owner, 2026-06-21 | See §9 |
--
92-- A green CI / DONE status proves **key-parity + render + no-bare-strings**, **not**
translation *quality* ? that's human-gated and owner-paced.
93-- Kannada/Hindi will **not render** until the Noto fonts land (D-FONT); until then kn/hi
catalogs exist but display tofu/fallback.
94-- The SPA is **job-tethered**; localizing **dynamic engine errors/report text** needs engine
P2 (owner-gated, cross-repo, D-BACKEND-DEPTH) ? out of this DoD.
95-- The query bar stays **English-grammar parsing**; kn/hi users rely on tap-only controls (a
later, larger slice), not free-text Kannada queries.
96-- **Mobile-*perfect*** layout is an alpha non-goal (usable-on-phone is in scope; pixel-
perfect is not).
97-
98-## 6.5 Testing strategy ? automatic language negotiation (rigorous, owner-required)
99-**The bar (owner, #41):** a user whose **computer/browser language is Kannada or Hindi must
get the respective version automatically** ? that is the core promise and it is **proven by
tests at every layer, never assumed.** Auto-selection IS the negotiation order (persisted pref ?
`?lang=` ? browser/`Accept-Language` ? `en`); each step gets a test.
100-
101-**Negotiation matrix ? every row is an automated test:**
--
117-- **Static site (B3):** the shared `t()` shim's negotiation is unit-tested the same way;
plus a server test that `Accept-Language: kn/`?`lang=kn` returns the kn-rendered page.
118-- **CI key-parity:** guarantees every locale has every key, so a correctly-negotiated locale
can never fall through to a raw key.
119-- **End-to-end (manual, owner machine ? per I18N_PLAN §6):** set the real OS/browser to
Kannada, then Hindi ? load both surfaces ? confirm the right language renders **and the Noto
fonts display** (the cloud CI container can't render fonts; the owner eyeball is the final
gate).
120:[Omitted long matching line]
121-
122-A locale that's reachable by the switcher but **not auto-selected from the browser language
is a bug, not a "later".** This matrix is a DoD gate (§9).
123-
--
127-| ID | Deliverable | Workstream | Depends on | Gated? | Status | PR |
128-|---|-----|-----|-----|-----|-----|----|
129-| P0 | This tracked project doc + inbound pointers | ? | ? | ? | No | ? |

```

```

[#41](https://github.com/avidullu/khelsutra/pull/41) · engine
[#275](https://github.com/Khelsutra/badminton-highlight-indexer/pull/275) |
130-| B1 | SPA i18n scaffold + language switcher, en-only (react-i18next + detector + `en`
catalog + switcher + persistence + jsdom render tests + browser-language negotiation
tests $6.5 N1/N3?N7) | B | P0 | No | ? | [#42](https://github.com/avidullu/khelsutra/pull/42)
|
131-| B2 | Extract SPA strings ? `en` catalog (~50 literals + ~16 derived; ICU plural keys); add
key-parity + no-bare-strings CI | B | B1 | No | ? |
[#45](https://github.com/avidullu/khelsutra/pull/45) |
132-| A1 | Plain-language copy pass on the `en` catalog (de-jargon; icon+text; non-color
selection) | A | B2 | No | ? | [#60](https://github.com/avidullu/khelsutra/pull/60) |
133-| B3 | Tiny shared `t()` shim + `data-i18n` plumbing for `site/`, en-only (proves shared
catalogs across surfaces) | B | B2 | D-CATALOG-HOME | ? | ? |
--
141-| A5 | Friendly engine-error mapping over raw FastAPI `detail` (SPA, client-side map) | A |
B2 | No | ? | [#62](https://github.com/avidullu/khelsutra/pull/62) |
142-| B6 | Hindi (`hi`) catalog ? render (the extensibility proof ? just a catalog) ? SPA
done (AI-draft, native review pending) | B | B5 | D-LOCALES ? | ? |
[#48](https://github.com/avidullu/khelsutra/pull/48) |
143-| A8 | Background-processing reassurance + verdict/status as icon chips (SPA) | A | B2 | No
| ? | [#63](https://github.com/avidullu/khelsutra/pull/63) |
144-| A9 | Tap-only filter controls alongside the English query bar (SPA) | A | B2 | No | ? | ?
|
145-| B7 | *(deferred, cross-repo)* Engine P2 backend customer-facing localization (report
`customer_` + rejection reasons) | B | engine | D-BACKEND-DEPTH, owner-gated | ? | ? |
146-
147-[Omitted long context line]
--
155-5. D-CATALOG-HOME ? ? ADOPTED ? co-locate `locales/` + CI in `khelsutra` (a shared i18n
dir consumed by both the SPA and the site). (The engine `I18N_PLAN` assumed the indexer repo;
this is the adaptation.)
156-
157-## 9. Definition of done
158- [ ] Automatic language negotiation ($6.5) is green ? a Kannada browser gets Kannada
and a Hindi browser gets Hindi (N1/N2); persisted-pref / `?lang` / region-subtag /
unsupported-fallback / missing-key-fallback all proven (N3?N7) on the SPA, and the static-site
shim honors `Accept-Language` + `?lang` (N8).
159- [ ] Manual cross-check (owner machine): OS/browser set to Kannada and to Hindi each
render the respective language on BOTH surfaces, with Noto fonts displaying.
160- [ ] `en + kn + hi` catalogs live and rendering on BOTH `web/` SPA and `site/`
marketing (switcher + persistence + `en` fallback; no raw keys shown).
161- [ ] Noto Sans Kannada + Devanagari render correctly ? owner eyeball on a real machine.
162- [ ] CI key-parity + no-bare-strings green; a stub locale flags every gap.
163- [ ] Recording-education (capture + homepage gist) available in `kn` + `hi`.
164- [ ] Non-tech UX baseline: plain-language copy (no raw engine jargon in user-facing
strings), mobile-usable touch targets, icon+text, WhatsApp share, honest today-vs-roadmap
preserved through translation.
165- [ ] $7 all rows ? (or explicitly deferred with a reason) ? flip Status ? DONE +
completion note ? run engine `DOC_STATUS.md $2` archival checklist ? `git mv` to
`docs/archives/`.
166-
167-## 10. References
168-[Omitted long context line]
--
170----
171-
172-### Changelog
173- `2026-06-21` ? created. Isolated the UX + Hindi/Kannada work into one tracked project
(owner directive). Grounded in a multi-agent research+verification pass (web/+site/ string maps,
Indic-l10n prior art, 5 adversarial verdicts). Decisions D1?D5 locked; 5 product/licensing
decisions surfaced as open; $7 tracker seeded with B1 as the recommended first slice.
174- `2026-06-21` ? owner locked D6 (en+kn?hi) + D7 (OFL Noto carve-out) and adopted
D8 (SPA-first) + D9 (catalogs in khelsutra); only D-BACKEND-DEPTH remains open. B1
greenlit ? ? in progress.
175- `2026-06-21` ? B1 opened ? [#42](https://github.com/avidullu/khelsutra/pull/42): SPA

```

i18n scaffold (react-i18next + ICU + detector) + `en` catalog + language switcher + the SPA's first jsdom render tests. Verified typecheck/79-tests/build green + live in-browser (switcher persists, en-fallback). Doc/pointer PRs opened: khelsutra [#41](https://github.com/avidullu/khelsutra/pull/41), engine [#275](https://github.com/Khelsutra/badminton-highlight-indexer/pull/275).

176:- `2026-06-21` ? \*\*\$6.5 Testing strategy added (owner ask, #41):\*\* a rigorous \*\*automatic-negotiation matrix\*\* (N1?N9) so a Kannada/Hindi \*\*browser\*\* auto-gets the respective version ? proven at the detection level now (B1), at the content level with the catalogs (B5/B6), on the static shim (B3), and by a manual font-render eyeball. Wired into the DoD (\$9). B1 (#42) gains the detection tests (N1/N3?N7).

177:[Omitted long matching line]

178-[Omitted long context line]

179:[Omitted long matching line]

180-[Omitted long context line]

## 27. assistant (2026-06-21T17:19:29.024Z)

I have everything I need. The faithfulness anchor is confirmed: \$6.4 caveat line 95 ("kn/hi users rely on tap-only controls") and \$7 row A9 ("Tap-only filter controls alongside the English query bar (SPA)"). A9 satisfies both ? tap controls render alongside the QueryBar, build ParsedQuery directly without parsing English, localized for all 6 locales.

Let me do one final check on the redundancy question ? confirm there's no `data-testid` collision and look at how the two chip rows actually stack visually, plus confirm the `tap.over10` "Over 10 seconds" matches the parser's `over 10s` semantics (the parser uses `>`, label says "Over 10 seconds" which is honest for `>10`).

Let me verify one subtle faithfulness/honesty point: the QueryBar EXAMPLES include `"first 10"` and `"top 15 longest"` which TapFilters does NOT offer ? and TapFilters offers `top5`/`top10` which the examples don't. This is a genuine differentiation, not pure duplication. Let me confirm the visual order in App.tsx (QueryBar examples row, then TapFilters row right below).

I've already confirmed from App.tsx (lines 249-253): `` then `` directly below. QueryBar's last visible element before TapFilters is the `query.support` paragraph (line 104-106), and its examples row is at line 90-102. So the visual stack is:

1. Input + Apply
2. Parsed pill
3. "Sample queries:" + English chips (`over 10s`, `top 15 longest`, `shortest first`, `first 10`)
4. Support note
5. "Or tap to filter:" + localized chips (Longest first, Shortest first, Over 10 seconds, Over 20 seconds, Top 5 longest, Top 10 longest)

Now I have a complete, precise picture. Let me compose findings.

## 28. user (2026-06-21T17:20:02.692Z)

Structured output provided successfully

## 29. assistant (2026-06-21T17:20:16.835Z)

A9 review complete. Summary of what I verified and found:

**\*\*Verified locally\*\*** (reproduced, not trusted from the description):

- 173 web tests pass; `tsc --noEmit` clean.
- The no-bare-strings hygiene test **\*\*does\*\*** cover TapFilters (`hygiene.test.ts:107` lists `components/TapFilters.tsx` in its FILES array).
- The tap==type test pins `parseQuery(f.english)` deep-equals `f.parts` for every control.
- Faithfulness: A9 meets \$6.4 ("kn/hi users rely on tap-only controls") and \$7 row A9 ("tap-only filter controls alongside the English query bar"). Nothing essential is missing ? reset/clear is covered by the existing Select-all bulk control.

**\*\*Findings** (all minor/nit ? no blockers):

1. **Redundancy** (minor, the one you flagged). **QueryBar's** `EXAMPLES` chips render **hardcoded English** literals (`QueryBar.tsx:99` renders `{ex}` directly, not a `t()` key), so a Kannada/Hindi user gets an untappable-by-meaning English chip row sitting directly above the localized A9 row ? exactly the syntax wall A9 routes around. They're not pure duplicates (EXAMPLES has `top 15 longest`/`first 10`; A9 has `Top 5`/`Top 10`/`Over 20s`), so a blind merge would drop functionality. **Recommendation:** hide the QueryBar "Sample queries:" row when `i18n.language !== 'en'` (and/or differentiate copy: "Type a query (examples):" vs "Tap to filter:").

2. **Copy nit:** `tap.title` "Or tap to filter:" depends on the row above it; the leading "Or" dangles if rows wrap apart on a phone. Replicated across all 6 locales. Drop the "Or".

3. **Trace nit:** taps log `raw: "tap:tap.longest"` (double `tap.` prefix, leaks the `i18n` key into the observability trace). Consider `f.english` instead so tap/type traces are comparable.

4. **Test-coverage nit:** the strong table test pins the parser vs `f.parts`, but the component builds its **own** `ParsedQuery` inline (`TapFilters.tsx:47`), and only `over10` is verified end-to-end. A hand-wiring slip on `top5`/`top10` wouldn't be caught. Loop the apply test over all `TAP_FILTERS`.

ally is clean: real `<button>`'s, `min-h-11` (?44px) verified by a test, localized text labels, icon-free.

The localized `tap.` sets for `kn/hi/es/da/id` are honest and accurate (e.g. "Over 10 seconds" honestly reflects the parser's `>10`; `kn/hi/es/da/id` all read naturally and preserve the length-led framing).